



Artificial Intelligence

(Course Code: CSC262)

Name : Muhammad Abdullah

Roll No # FA24-BSE-042

Instructor Name : Sir Waqas

Assignment # 1

Question #01:

1. Problem Statement

The person wants to **reach the airport** but:

- Has **never been there before**.
- Has **no external guidance** (no one awake, no GPS, etc.).
- It's **early morning**, so environment is quiet/empty.

So, the **problem** is:

The person needs to find the correct route from his current location (home) to the city airport **without prior knowledge** of the way.

2. Operators

These are **possible actions** the person can take to move toward the goal.

Examples:

1. **Move forward** on the current road.
2. **Turn left** at an intersection.
3. **Turn right** at an intersection.
4. **Go back** (if the chosen road leads nowhere).
5. **Observe** for road signs (if any exist).

Each operator **changes the current state**; i.e., moves the person from one location (state) to another.

3. Solution Space

The **solution space** is the **set of all possible routes** the person can take from his house to the airport.

In other words:

- Each **state (node)** represents a **location** (e.g., home, intersection, road segment).
- Each **operator (edge)** represents a **possible move** between locations.

So, the **solution space** includes:

- All roads and intersections between the home and the airport.

- All possible sequences of moves (left, right, straight, back) that could potentially reach the airport.

Formally:

Solution Space = { all paths the driver can take from start → airport }

4. Goal State

The **goal state** is clearly defined as:

The person reaches the **city airport**.

That means the search stops when the **current location = airport**.

5. Search Strategy

There are two main strategies:

- **Blind (uninformed) search** – no prior knowledge of the route.
- **Heuristic (informed) search** – uses some knowledge or estimation (e.g., distance, direction).

Since:

- The person has **no prior knowledge** of the road network.
- There's **no map or GPS**.
- He must **explore blindly**.

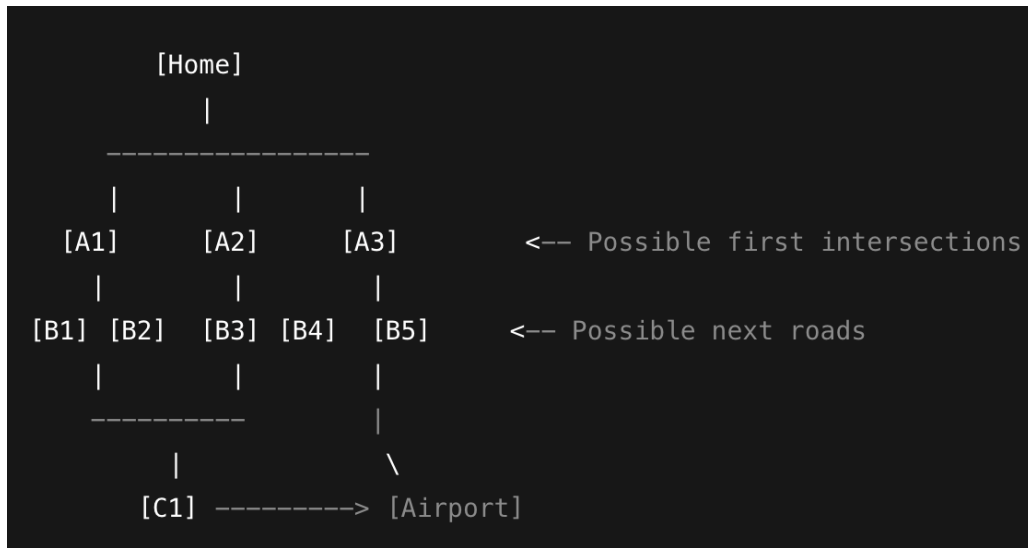
Therefore, **he should follow a Blind Search Strategy**, such as:

- **Depth-First Search (DFS)** (keep driving in one direction until dead-end, then backtrack), or
- **Breadth-First Search (BFS)** (try all nearby routes systematically).

If he *had* access to a map or some directional hint (e.g., “airport is north”), then a heuristic search (like A)* would be better.

6. Graphical Representation (Conceptual Map)

Let's model this in a **state-space graph**:



Explanation:

- Each node (A1, B1, etc.) = a **location or intersection**.
- Each line = a **possible road (operator)**.
- The person starts at **Home** and explores the network of nodes.
- The **search continues** until the **node = Airport** (goal state).

Question #02:

1. Definition & Comparison

Feature	Well-Structured Problem (WSP)	Ill-Structured Problem (ISP)
Nature of the problem	Clearly defined and organized.	Vague, unclear, or ambiguous.
Initial state	Clearly known.	Not clearly defined.
Goal state	Clearly specified and measurable.	Uncertain or subjective.
Operators / Rules	Fixed, well-defined rules and methods.	No fixed or agreed-upon methods.
Solution path	Step-by-step procedures can be applied.	Requires reasoning, creativity, and judgment.
Outcome	Single, correct, and verifiable solution.	Multiple possible solutions, often none “perfect.”
Domain	Structured environments (e.g., math, logic, programming).	Real-world or social domains (e.g., business, politics, design).

2. Examples

Example 1: Well-Structured Problem (WSP)

Mathematics or Logic Problem

- **Problem:** Solve for x in the equation $2x + 4 = 10$.
- **Characteristics:**
 - Initial state: $2x + 4 = 10$
 - Goal state: Find x
 - Rules: Apply algebraic operations
 - **Solution:** $x = 3$
There is a **clear procedure and a single correct answer**.

Example 2: Ill-Structured Problem (ISP)

Real-world Strategic Problem

- **Problem:** How can Pakistan improve its education system?
- **Characteristics:**
 - Initial state: Current education challenges are complex and multifaceted.
 - Goal state: “Improved education system”, vague and subjective.
 - Rules: No fixed steps; multiple strategies possible.
 - **Solutions:** Vary based on perspective; policy reform, teacher training, digital tools, etc.

There’s **no single correct answer**, and it involves **judgment and creativity**.

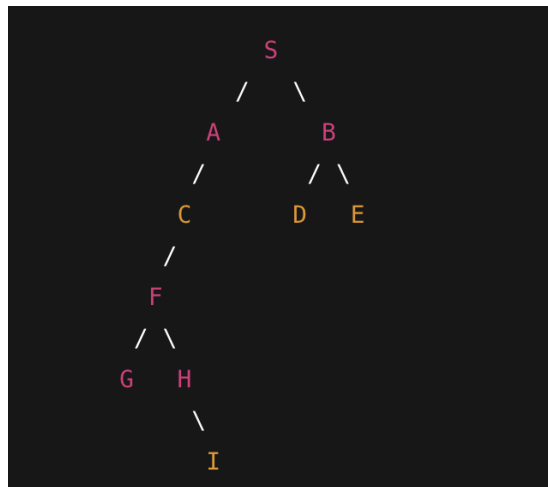
Another Pair for Clarity

Well-Structured	Ill-Structured
Solving a Sudoku puzzle.	Designing a new city layout.
Writing a sorting algorithm.	Creating a startup business plan.
Calculating compound interest.	Improving public transport in a crowded city.

3. Key Insight

- WSPs are ideal for **computers, algorithms, and structured reasoning**.
- ISPs require **human intuition, experience, creativity, and critical thinking**.

Question #03:



Start State: S

Goal State: D

1. BFS (Breadth First Search)

BFS uses a **Queue (FIFO)**, explores level by level.

Step	Queue (Q)	Visited	Action
1	[S]	[]	Start from S
2	[A, B]	[S]	Expand S → Enqueue A, B
3	[B, C]	[S, A]	Expand A → Enqueue C
4	[C, D, E]	[S, A, B]	Expand B → Enqueue D, E
5	[D, E, F]	[S, A, B, C]	Expand C → Enqueue F
6	[E, F]	[S, A, B, C, D]	Goal D found → Stop

BFS Order:

S → A → B → C → **D**

Goal found at Level 2.

2. DFS (Depth First Search)

DFS uses a **Stack (LIFO)**, goes deep before backtracking.

Step	Stack (Q)	Visited	Action
1	[S]	[]	Start from S
2	[B, A]	[S]	Expand S → Push A, B (Rightmost last if standard left-to-right)
3	[B, C]	[S, A]	Expand A → Push C
4	[B, F]	[S, A, C]	Expand C → Push F
5	[B, H, G]	[S, A, C, F]	Expand F → Push G, H
6	[B, H]	[S, A, C, F, G]	Visit G (no children)
7	[B, I]	[S, A, C, F, G, H]	Expand H → Push I
8	[B]	[S, A, C, F, G, H, I]	Visit I (no children)
9	[]	[S, A, C, F, G, H, I, B]	Expand B → Push D, E
10	[E]	[S, A, C, F, G, H, I, B, D]	Goal D found → Stop

DFS Order:

S → A → C → F → G → H → I → B → **D**

Summary

Search	Data Structure	Nodes Expanded (Order)	Goal Found At
BFS	Queue (FIFO)	S, A, B, C, D	Level 2
DFS	Stack (LIFO)	S, A, C, F, G, H, I, B, D	Level 4

Question #04:

Progressive Deepening Search (IDS)

Progressive deepening (or **iterative deepening search**) is a combination of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

It's designed to remove the disadvantages of both while keeping their advantages.

How It Mixes DFS and BFS

- From **DFS**, it takes **low memory use**, it goes down one path at a time.
- From **BFS**, it takes **completeness and optimality**, it will eventually find the goal at the shallowest depth.
- The idea is to **run DFS repeatedly**, but each time with an **increased depth limit**.

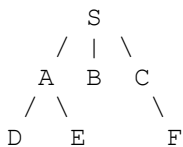
So first it searches to depth 0, then 1, then 2, and so on until the goal is found.

Why It's Better

Issue	DFS	BFS	Progressive Deepening
Memory use	Very low	Very high	Low (like DFS)
Completeness	Can get stuck in infinite paths	Always complete	Complete
Finds shortest goal	Not guaranteed	Yes	Yes
Works well when depth unknown	No	No	Yes

Example

Let's say we have this tree:



Goal = **F**

Depth 0: Search only node **S** → not found

Depth 1: Search **S, A, B, C** → not found

Depth 2: Search **S, A, D, E, B, C, F** → goal found

So it behaves like BFS (finds the shallowest goal) but uses DFS each time, that's why memory stays small.

In Simple Words

- **DFS** can go too deep and miss the goal.
- **BFS** uses too much memory.
- **Progressive deepening** starts small and grows, it's like **doing DFS in layers**. It ensures the solution is found at the **minimum depth** and with **very low memory**.

Summary Table

Algorithm	Search Type	Memory	Completeness	Optimal
DFS	Depth-first	Low	No	No
BFS	Breadth-first	High	Yes	Yes
IDS	Iterative deepening	Low	Yes	Yes

Question #05:

A. Problem definitions (precise)

We assume a **minimization** hill-climbing where a node's number = estimated remaining distance to goal (smaller is better).

1. **Local optimum (local minimum)**
A node where **all immediate neighbors are worse or equal** (no neighbor has strictly lower heuristic). Greedy HC stops here even if a better global solution exists elsewhere.
2. **Plateau**
A region where **many neighbors (or the node and neighbors) have equal heuristic**. Greedy choice gives no improvement; algorithm may stall or wander.
3. **Ridge**
The true path to the global optimum requires **temporary moves that increase the heuristic** (or require alternating moves). A purely greedy algorithm rejects these temporary worse moves and cannot climb the ridge.

B. Practical solutions (what to use & when)

List of methods with concise trade-offs:

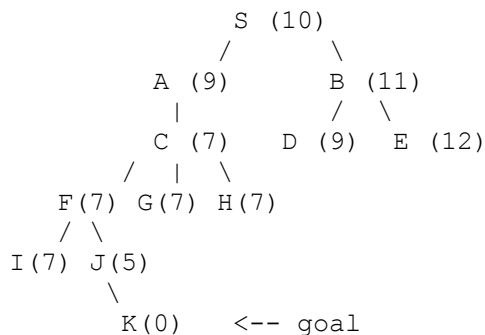
1. **Steepest-ascent (best-improvement)**
 - Evaluate **all** neighbors; pick the neighbor with the lowest heuristic.
 - Pros: avoids poor tie-breaks, often better than pure greedy.
 - Cons: more computation per step.
2. **One-step / limited lookahead**

- When neighbors tie, inspect their children (or depth-2) and prefer nodes whose descendants show improvement.
- Pros: cheap way to break plateaus/ties.
- Cons: extra branching evaluation.
- 3. **Allow limited sideways moves**
 - Permit moves to neighbors of equal heuristic up to a fixed **sideways_limit**.
 - Pros: cross plateaus without full restarts.
 - Cons: could wander; needs limit.
- 4. **Random tie-breaking + random restarts**
 - If stuck, restart from a new random start (or random neighbor).
 - Pros: simple and effective across many landscapes.
 - Cons: repeated restarts may be costly.
- 5. **Stochastic hill-climbing / simulated annealing**
 - Occasionally accept worse moves with probability $p(t)$ (decreasing).
 - Pros: escapes local optima & ridges robustly.
 - Cons: requires tuning; not deterministic.
- 6. **Hybrid with memory/backtracking (tabu, limited DFS)**
 - Keep a small visited list or allow controlled backtracking to previously visited nodes.
 - Pros: avoids immediate cycles and supports recovery.
 - Cons: increases memory.
- 7. **Switch to a different search** (e.g., A*, beam search)
 - If heuristic is unreliable, choose an algorithm using path-cost or stronger heuristics.

Practical recommendation: Start with **steepest ascent + one-step lookahead + limited sideways**. If still failing, use **random restarts** or **simulated annealing**.

D. Correct tree (exact structure used here)

I'll restate the correct tree and labels (minimization: smaller = closer to K):



E. Apply hill-climbing

1) Plain greedy hill-climbing (single-step, deterministic tie-break leftmost)

- **Start:** S(10)
neighbors: A(9), B(11) → pick **A(9)** ($9 < 10$)
- **A(9)** → neighbors: S(10), C(7) → pick **C(7)**
- **C(7)** → neighbors: A(9), F(7), G(7), H(7)
best = 7 (tie F/G/H). Naive tie-break picks **F(7)** (leftmost) → luck: good
- **F(7)** → neighbors: C(7), I(7), J(5) → pick **J(5)**
- **J(5)** → neighbors: F(7), K(0) → pick **K(0)** → **Goal reached**

Lucky path (naive): $S \rightarrow A \rightarrow C \rightarrow F \rightarrow J \rightarrow K$

But if tie-breaking had returned G or H at C, those may be dead ends (no <7), and greedy would **stop** at $h=7$, plateau/local minimum.

Therefore plain greedy is unreliable because of the plateau at C.

2) Apply robust variant: *Steepest-descent + 1-step lookahead + sideways_limit=1*

Rationale: steepest-descent avoids poor local tie decisions; lookahead breaks plateau by checking if children of F/G/H lead to improvement.

Step trace:

- **S(10)** → A(9) chosen (best neighbor).
- **A(9)** → C(7) chosen.
- **At C(7):** neighbors F(7), G(7), H(7) tie on $h=7$. Use **1-step lookahead**:
 - Inspect children:
 - F's children: I(7), J(5) → J(5) indicates improvement exists below F.
 - G's children: assume none with <7 (dead end).
 - H's children: assume none with <7 .
 - **Select F** because its descendant J has $h=5$ (best lookahead).
- **F(7)** → pick **J(5)** (strict improvement).
- **J(5)** → pick **K(0)** → **Goal**.

Path: $S \rightarrow A \rightarrow C \rightarrow F \rightarrow J \rightarrow K$ (success).

3) If lookahead unavailable or expensive, use sideways moves + restarts

- At C, pick any equal neighbor; allow **sideways_limit = 2** moves. If you wander but find no improvement within limit, **restart** from a different start (or random neighbor). With a few restarts, probability of hitting F increases.

4) If problem were a ridge (not here), simulated annealing

- Accept temporary increases in h with probability $\exp(-\Delta h / T)$ with T decreasing. This allows climbing ridges where you must go uphill briefly to descend later.

F. Why these fixes work (short justification)

- **Steepest-descent** avoids unlucky tie-breaking because it considers *all* neighbors.
- **Lookahead** reveals promising branches behind plateaus without global restarts.
- **Sideways + restarts** are cheap and robust when branching is large but heuristics are weak.
- **Simulated annealing** handles ridges/local minima by accepting uphill moves probabilistically.

G. Complexity / practical tips

- **Time:** steepest-descent and lookahead cost more per node (evaluate b or b^d nodes), but avoid wasted restarts.
- **Space:** hill-climbing variants remain low-memory ($O(d)$), unlike BFS/A*.
- **Tuning:** `sideways_limit` $\approx 1-3$ usually enough; restarts $3-10$ depending on cost of restart; annealing schedule requires domain tuning.

Question #06:

How Best First Search Works in a Tree

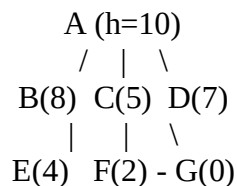
Best First Search (BFS) is an informed search algorithm that explores a tree by always expanding the node that appears most promising, based on a heuristic function $h(n)$.

Detailed Steps

- **Initialize a Priority Queue:** Start with the root node in a priority queue, ordered by the heuristic value $h(n)$.
- **Expand Nodes:** Remove the node with the lowest heuristic value from the queue. This is considered the 'best' node at that moment.
- **Generate Children:** Expand this node by generating its children and calculating their heuristic values.
- **Add Children to Queue:** Insert the children into the priority queue, maintaining the order by heuristic value.
- **Repeat:** Continue this process until the goal node is removed from the queue or the queue is empty.

Example Tree

Suppose you have the following tree, where each node is labeled with its heuristic value $h(n)$:



Search Process:

- Start at **A (10)**. Add B(8), C(5), D(7) to the queue.
- Choose **C (5)** (lowest h), expand to F(2).
- Choose **F (2)**, expand to G(0).
- Choose **G (0)**, which is the goal.
- **Path found:** $A \rightarrow C \rightarrow F \rightarrow G$.

Is Best First Search Always the Best Strategy?

- **No, it is not always the best strategy.** Best First Search relies solely on the heuristic $h(n)$ and ignores the actual cost already spent ($g(n)$). If the heuristic is misleading, the algorithm may follow a path that looks promising but is actually longer or more expensive.

- It can get stuck in local minima or take suboptimal paths if the heuristic does not accurately reflect the true cost to the goal.

Does Best First Search Guarantee the Best Solution?

- **No, it does not guarantee the best solution.** Since it does not consider the actual cost from the start node, it may miss the optimal path. Algorithms like **A*** search, which use $f(n)=g(n)+h(n)$, can guarantee optimality if the heuristic is admissible.

Table

Property	Best First Search	A* Search
Considers Actual Cost (g(n))	No	Yes
Uses Heuristic (h(n))	Yes	Yes
Guarantees Optimal Solution	No	Yes (if admissible)
Typical Usage	Fast, approximate	Optimal pathfinding

Question #07:

Beam Search is a **heuristic search algorithm** that explores a search tree by expanding only a fixed number of the most promising nodes at each depth (the "beam width" or degree of the search). The degree specifies how many nodes are kept and expanded at each level.

How Beam Search Degree 3 Propagates

- At each depth (or tree level), beam search evaluates all possible children of the current nodes.
- It then selects the top **3 nodes** (based on their heuristic scores) and discards the rest. Only these top 3 are expanded in the next step.
- This process repeats at every level of the tree, maintaining only the best 3 candidates (according to the heuristic) at each stage.

Example

If you start at node A, find all its children, and retain the three best (say B, C, and D), you will then expand only B, C, and D at the next level, and again select the top 3 from their children. The search continues by always keeping only 3 nodes per level, based on their heuristic scores.

Is Beam Search with Degree = 1 Equal to Best First Search?

- **Yes**, when the degree of beam search is set to 1, it always keeps only the single most promising node at each level.
- This means only one branch is explored at any time, using the heuristic to decide the path, which is exactly how **Best First Search** operates.

- So, **beam search with degree = 1 is equivalent to best first search** as both algorithms select and expand only the current best node at every step.

Question #08:

Main Concept: Branch and Bound Search

Branch and Bound is a search strategy used to solve optimization and decision problems, like pathfinding. The key idea is:

- **Branch:** Systematically explore the solution space by splitting it into subproblems or branches (i.e., paths from the start node).
- **Bound:** Use an evaluation function to calculate a lower bound (the best possible score from a partial solution). If this bound is worse than the best solution found so far, **prune** (stop searching) this branch.

This allows the algorithm to avoid unnecessary searches and focus only on promising paths, making it more efficient than brute force.

Simulating Branch and Bound on the Given Graph

Shortest path from S (leftmost node) to G (rightmost node).

Step 1: Start at S (Estimated Distance = 8)

- From S, possible next cities:
 - A via cost 2
 - D via cost 8

Step 2: Expand best node (lowest path cost + estimated distance)

- $S \rightarrow A$ (cost=2, A's estimate=6) : total estimate = $2 + 6 = 8$
- $S \rightarrow D$ (cost=8, D's estimate=4): total = $8 + 4 = 12$
- Choose $S \rightarrow A$ (estimate 8, smaller)

Step 3: Expand A

- $S \rightarrow A \rightarrow B$ ($2+1=3$, B's estimate=3): $3+3=6$
- $S \rightarrow A \rightarrow C$ ($2+3=5$, C's estimate=4): $5+4=9$
- $S \rightarrow A \rightarrow D$ ($2+2=4$, D's estimate=4): $4+4=8$
- Paths so far: $S \rightarrow A$ (estimate=8); $S \rightarrow A \rightarrow B$ (6); $S \rightarrow A \rightarrow C$ (9); $S \rightarrow A \rightarrow D$ (8); $S \rightarrow D$ (12)
- Best is $S \rightarrow A \rightarrow B$, with bound 6

Step 4: Expand $S \rightarrow A \rightarrow B$

- $S \rightarrow A \rightarrow B \rightarrow C$ ($3+3=6$, C's estimate=4): $6+4=10$
- $S \rightarrow A \rightarrow B \rightarrow E$ ($3+5=8$, E's estimate=3): $8+3=11$
- Paths: $S \rightarrow A \rightarrow B \rightarrow C$ (10); $S \rightarrow A \rightarrow B \rightarrow E$ (11)
- Other open paths: $S \rightarrow A \rightarrow C$ (9); $S \rightarrow A \rightarrow D$ (8); $S \rightarrow D$ (12)
- Now, best path is $S \rightarrow A \rightarrow D$, bound=8

Step 5: Expand $S \rightarrow A \rightarrow D$

- $S \rightarrow A \rightarrow D \rightarrow E$ ($4+1=5$, E's estimate=3): $5+3=8$
- Compare to $S \rightarrow A \rightarrow C$ (9), $S \rightarrow D$ (12), $S \rightarrow A \rightarrow B \rightarrow C$ (10), $S \rightarrow A \rightarrow B \rightarrow E$ (11)

Step 6: Expand $S \rightarrow A \rightarrow D \rightarrow E$

- $S \rightarrow A \rightarrow D \rightarrow E \rightarrow F$ ($5+1=6$), F's estimate=2: $6+2=8$
- $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$ ($5+3=8$), G's estimate=0: $8+0=8$

- Path $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$ is to goal with cost=8

Step 7: Look for better bound

- Remaining open paths have higher or equal estimated cost; no better solution is found.

Result

- **Optimal path found:** $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$ with total cost 8.

Suggest Improvements

- Use a better heuristic to make bounds tighter, resulting in more effective pruning (e.g., ensure estimates never overstate real costs).
- Combine with algorithms like A* (which always selects node with minimal $g(n) + h(n)$) for more reliable optimality and faster convergence.
- Memorization: Track the best cost to each node, prevent expanding worse duplicate paths.

Question #09:

Leaf scores (from left to right):

80, 10, 55, 45, 65, 100, 20, 35, 70

The Minimax algorithm alternates levels between the maximizing player (Max) and the minimizing player (Min):

- **Assume the root node is Max.**
- Its children are Min nodes, whose children are Max nodes (leaf values).

Move level by level, from the leaf nodes upwards.

1. Move up one level (Max nodes at leaves):

Each Min node chooses the minimum value among its children (from left to right):

- **First Min node:** children 80, 10
 $\text{Min}(80, 10) = 10$
- **Second Min node:** children 55, 45, 65
 $\text{Min}(55, 45, 65) = 45$
- **Third Min node:** children 100, 20
 $\text{Min}(100, 20) = 20$
- **Fourth Min node:** children 35, 70
 $\text{Min}(35, 70) = 35$

So now, at the Min level above the leaves, the scores are: **10, 45, 20, 35**

2. Move up to the next level (Max chooses the maximum of its Min children):

- $\text{Max}(10, 45, 20, 35) = 45$

3. Root node value

The minimax value of the root is **45**. This means if both players play optimally, the best achievable value starting from the root is 45.

Level	Node type	Values computed
Leaves	Static	80, 10, 55, 45, 65, 100, 20, 35, 70
Above leaves	Min	10, 45, 20, 35
Above that	Max (root)	45

Question #10:

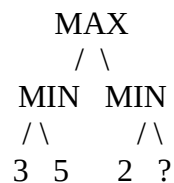
Alpha-Beta Pruning is an enhancement to the minimax search algorithm that reduces the number of leaf nodes evaluated by the algorithm. It does this by "pruning" (ignoring) branches that cannot possibly influence the final decision, allowing the algorithm to search more efficiently.

How Alpha-Beta Pruning Minimizes Evaluations

- **Alpha (α):** The best value that the maximizer can guarantee so far.
- **Beta (β):** The best value that the minimizer can guarantee so far.
- While traversing the tree, if you discover that a move cannot improve upon the best already found for either player, you stop evaluating further moves in that branch.
- This avoids wasting time on evaluating parts of the tree that will not be chosen during optimal play.

Example 1

Simple minimax tree:

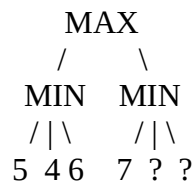


Suppose we already evaluated the left MIN child (3, 5). MIN chooses **3** (the lower value; $\beta = 3$ for this branch). Now, MAX knows its left side is worth at least 3. When evaluating the right MIN child, we look at '2' first. If '2' is less than the current $\alpha = 3$, then MAX would never pick this branch (since $2 < 3$), so there's no need to evaluate further offspring (the '?' node is ignored).

Pruning occurs: We do not evaluate the '?' leaf. Only three leaf evaluations are made instead of four!

Example 2

Suppose we have:



- First, evaluate 5, 4, 6 under the left MIN. MIN's best is **4**.

- Now, evaluating right MIN branch: check '7'. As soon as $7 > 4$ ($\alpha = 4$ for MAX so far), MAX has a better choice already, so the rest ('?' and '?') under this MIN branch are *pruned* (not evaluated).

Question #11

Step 1: Assign Leaf Values

Leaves (from left to right): 30, 50, 40 | 70, 60, 20 | 80, 90 | 10, 90 | 70, 60

Step 2: Evaluate Bottom Maximizing Level

For each maximizing node at the bottom:

- **Node 1:** $\max(30, 50, 40) = 50$
- **Node 2:** $\max(70, 60, 20) = 70$
- **Node 3:** $\max(80, 90) = 90$
- **Node 4:** $\max(10, 90) = 90$
- **Node 5:** $\max(70, 60) = 70$

Step 3: Evaluating Middle Minimizing Level (Alpha-Beta Pruning Applied)

We now move up to minimizer nodes:

- **Node A:** $\min(50, 70) = 50$
- **Node B:** $\min(90, 90) = 90$
- **Node C:** $\min(70) = 70$

Pruning:

Alpha-Beta Pruning

As we process each minimizer node:

1. At Node A: First child has value 50. **Beta = 50.**
2. At Node A: Next child has value 70. Beta stays 50 (since $70 > 50$).
 - If any further child had a value less than 50, update Beta; otherwise, Beta remains 50.
3. At Node B: First child has 90. Beta = 90. Next child 90. No update: Beta remains 90.
4. At Node C: Only one child. Beta = 70.

In this example, **no pruning occurs**, because each minimizer only has two children, or the best value isn't realized until both are checked.

Step 4: Compute Root Maximizer Value

The root node takes the max of its children:

- $\max(50, 90, 70) = 90$

Table

Node Type	Children	Value Computed
Maximizer (bottom)	30, 50, 40	50
Maximizer (bottom)	70, 60, 20	70
Maximizer (bottom)	80, 90	90
Maximizer (bottom)	10, 90	90
Maximizer (bottom)	70, 60	70
Minimizer (middle)	50, 70	50
Minimizer (middle)	90, 90	90
Minimizer (middle)	70	70
Maximizer (root)	50, 90, 70	90